# NPCYF Intent Classifier

System Architecture and Technical Documentation

**Pulkit Chouhan**

February 26, 2026

# Contents


**1 Introduction** **3**

**2 Multiclass Intent Classifier Architecture** **3**
- 2.1 Model Construction . . . . . 3
- 2.2 Intent Classes . . . . . 3
- 2.3 Production Integration . . . . . 3

**3 Configuration Management (`config.json`)** **4**
- 3.1 Structure . . . . . 4
- 3.2 Execution Lifecycle . . . . . 4

**4 Core Scripts** **4**
- 4.1 `interactive_inference.py` . . . . . 4
- 4.2 `benchmark_sql_modular.py` . . . . . 5

**5 Project Directory Structure** **5**

# 1 Introduction

The **NPCYF Intent Classifier** is a robust multi-class NLP routing engine designed to categorize user queries relating to crop yield forecasting and climatic data. By analyzing the structural and semantic components of an incoming text prompt, the system routes the request to the most appropriate downstream processing pipeline: an embedded Database Query Generator (Text-to-SQL), a Conversational Assistant, or Platform Knowledge Base retrieval.

# 2 Multiclass Intent Classifier Architecture

At the core of the system sits a high-performance machine learning classifier built on top of the `scikit-learn` framework.

## 2.1 Model Construction

The training pipeline (`train_multiclass_model.py`) dynamically constructs a sequential ML Pipeline consisting of two primary stages:

1. **Text Vectorization (TfidfVectorizer)**: Transforms raw text into a matrix of TF-IDF features using Unigrams and Bigrams (`ngram_range=(1,2)`), effectively capturing local word contexts while stripping standard English stop words.

2. **Classification (LogisticRegression)**: Uses a Multinomial Logistic Regression model optimized via the `lbfgs` solver allowing for robust probability estimations across all competing classes simultaneously.

## 2.2 Intent Classes

The model is explicitly configured to distinctively recognize **three (3)** operational classes:

- **`sql`**: Technical queries that demand real-time data retrieval from the underlying PostgreSQL schema (e.g., *"Show me rainfall data for Bihar"*).

- **`general`**: Conversational, domain-agnostic inquiries routed to localized Large Language Models (LLMs) (e.g., *"What is the yield of wheat?"*).

- **`platform`**: Instructional questions regarding the usage of the NPCYF platform itself, routed typically to internal documentation (e.g., *"How do I create a dataset in NPCYF?"*).

## 2.3 Production Integration

During runtime, the `MulticlassIntentClassifier` wrapper located in `src/` loads the deserialized pickle model (`.pkl`). It generates probability distributions, extracts the highest-confidence match, and enforces an environment-variable-driven fallback threshold (`INTENT_THRESHOLD`). If confidence dips below this configurable value, queries default to `general` ensuring safe failure modes.

# 3 Configuration Management (`config.json`)

The `config.json` file acts as the central orchestrator for model pathways and operational parameters across the deployed environment.

## 3.1 Structure

```
{
  "models": {
    "general": {
      "path": "models/general_model.gguf",
      "n_ctx": 2048,
      "n_gpu_layers": -1
    },
    "sql": {
      "path": "models/qwen2.5-coder-1.5b-instruct-q4_k_m.gguf",
      "n_ctx": 8192,
      "n_gpu_layers": -1
    },
    "intent_classifier": {
      "path": "models/intent_model_multiclass.pkl"
    }
  },
  "paths": {
    "schema": "resourcesfortrainingtheintentclassifier/Database
     Schema.sql"
  }
}
```

Listing 1: Sample config.json configuration

## 3.2 Execution Lifecycle

The JSON file is natively parsed by execution scripts (`interactive_inference.py`, `benchmark_sql_modular.py`). By decoupling hard-coded file locations from Python logic, DevOps engineers can seamlessly hot-swap quantized local models (‘.gguf‘) or update context window constraints (`n_ctx`) without altering foundational source code.

# 4 Core Scripts

## 4.1 `interactive_inference.py`

This is the primary user-facing conversational loop operating entirely within the local terminal. Its workflow comprises:

- **Credentials & Bootstrapping**: Inherits PostgreSQL secrets dynamically via `.python-dotenv`, or queries the operator securely. Connects and pulls authoritative “State” and “District” catalogs directly from the Database.

- **Routing**: Feeds user inputs into the Intent Classifier, recording classification latencies.
- **Dynamic Prompting (SQL Path)**: Detects if the user mentioned a specific "State" or "District" bounding box. It utilizes a 'SchemaValidator' to prune irrelevant schema tables entirely off the LLM prompt, forcing the GGUF SQL Coder Model into high-accuracy, spatially restricted CTEs.
- **Execution & Auto-Repair**: Injects generated SQL into Postgres. Utilizing bespoke regex parsers, it intercepts syntactic anomalies (like non-aliased aggregates), queries the Database natively, and initiates autonomous validation/correction 'Repair Prompts' back to the LLM if anomalies arise.

### 4.2 `benchmark_sql_modular.py`

This utility empowers CI/CD pipelines to quantitatively evaluate SQL generation updates against Gold Standard queries (`input.csv`).

- **Execution Paradigm**: Iterates through target prompts, dynamically instantiating LLM environments, executing derived queries, and sweeping GPU VRAM aggressively (`gc.collect()`) to mitigate memory bleed across 100+ prompt suites.
- **Fuzzy Evaluation**: It determines execution validity not by strict string matching, but by evaluating the generated numeric arrays against floating-point gold variances (e.g., $abs(G - M) < 0.01$). Matches and execution velocities are dumped into `output.csv`.

## 5 Project Directory Structure

The system layout promotes modular separation of concerns:

```
intent_classifier/
├── benchmark/ ...  I/O directories for SQL regression
│                   testing matrices
├── config.json ...  Central configuration registry
├── data/ ...  Assorted static evaluation sets
├── models/ ...  Destination vault for downloaded
│                GGUF & Pickle files
├── requirements.txt ...  Strict virtual environment
│                         dependency lock
├── resourcesfortrainingtheintentclassifier/ ...  Training CSVS and Schema definitio
├── scripts/ ...  Execution binaries
│   ├── benchmark_sql_modular.py
│   ├── interactive_inference.py
│   └── train_multiclass_model.py
└── src/ ...  Core logical classes
    └── multiclass_intent_classifier.py
```